

Programming Assignment 2: continuity

COP 3223, Fall 2024

Due: September 20th, 2024 before 11:59 PM

Contents

Objective	2
1 Motivation	3
2 Requirements	3
3 Adopting a Growth Mindset	3
4 Function Requirements	3
4.1 Main Function	4
4.2 Determining Max Items	4
4.3 Grade Calculator	4
4.4 Shakespeare Email	5
4.5 Skeleton Code	6
5 Special Restrictions	6
5.1 No Global Variables	6
5.2 No File I/O	7
5.3 IDEs	7
5.4 Linux/Mac Command Line	7
6 General Tips for Working on Programming Assignments	8
7 Deliverables (Submitted via Webcourses, not Github Classroom)	8
7.1 Example Output	9
8 Grading	9

Objective

In this assignment, you will write a program that emulates a river found in the mountain. This type of river is consistent yet exuberant. It creates a continuous flow of water but contains lively energy in the form of rocks, fishes, bears, etc. Think of it like this, as you create iterations and loops, this is the river. However, as you create conditionals, these are the things you find **inside** the river.

We are attempting to establish a sense of continuity within programming and preparing for more complex applications. You do not want to forcefully create cracks that **hinder** a program's flow but rather, allow for fractures to naturally exist and work jointly with your application.

1 Motivation

Sometimes its hard to be as swift as the coursing river. For many of us, we tend to stay stuck on one thing and not allow everything around us to move and flow.

The biggest takeaway I want y'all to get from this assignment is simple: don't be bogged down by everything in the world.

Every problem, every incident, everything is only conditional within the short span of our lives. We, as humans, have the choice to **continue**, to **break**, or to simply ignore.

As you write your program with the necessary conditionals, remember, the program still moves on. As should you. Do not stay stuck in one iteration of your life when your success and happiness is conditionally dependent on your choices.

However, if you are unable to flow and there too many conditionals in your life, I encourage you to break from the moment. Breathe, and re-evaluate why these conditionals are the way that they are and if you can change them.

2 Requirements

You will submit a single source file, named **continuity.c**, that contains all required function definitions, and any helper functions you deem necessary.

3 Adopting a Growth Mindset

Excerpt from Dr. Szumlanski:

A word of advice before we dive in to the details: When faced with an assignment like this, which has many different facets, some of which might appear foreign and/or challenging, it's important not to look at it as an instrument that is being used to measure how much you already know (whether that's knowledge of programming, operating systems, or even what some might call "natural intellectual capability"). Rather, it's important to view this assignment as a chance to learn something new, grow your skill set, and embrace a new challenge. It's also important to view intellectual capability as something that can grow and change over one's lifetime. Adopting that mindset will allow you to reap greater benefits from this assignment (and from all your college-level coursework) regardless of the grades you earn.

4 Function Requirements

In the source file you submit, **continuity.c**, you must implement the following functions. You may implement helper functions to make these work, as well,

although that is probably unnecessary for this assignment. Please be sure the spelling, capitalization, and return types of your functions match these prototypes exactly.

4.1 Main Function

```
int main(int argc, char **argv);
```

- **Arguments:** This function should have the two defined arguments above. These are for autograding purposes.
- **Output:** This function should have no output.
- **Return:** A simple integer. Just 0.

4.2 Determining Max Items

Context: Every item has a base price. However, as someone struggling with money, the player will optimize their spending. They either bundle with other items or cheat the system with coupons. Here are the details:

- the first item starts at \$2
- each additional item is \$1 more (so the 2nd item is now \$3)
 - however, you have a 66% coupon whenever the item would be divisible evenly by 3
 - that means the 2nd item which would've been \$3 is now only \$1
 - the third item would be \$4 (as opposed to 2 since I never said 1 more based on the previous price)

```
determineMaxItems();
```

- **Arguments:** This function will have one argument
 - **maximumLimit:** an integer
 - This will be a scanned in value from the user outside of the function. It represents the amount of money you can spend
- **Output:** This function will output the following:
 - "I can only afford X items." where X is the calculated value based on the logic above
- **Return:** An integer representing the difficulty from 1 to 5
- **Disclaimer:** You must use a different looping structure than the ones used previously.

4.3 Grade Calculator

Context: Every student gets grades on their assignments. However, the system that the school uses, is not the best. Regardless of why that's the case, its up

to the student to make their own grading calculator. Here are the details:

- A student has a list of 8 grades. Each grade is out of 100 points.
- The first three items are worth 10% each of their total grade
- The next two items are worth 20% each of their total grade.
- The sixth and seventh are worth 5% each of their total grade.
- The eighth item is worth whatever is left.

determineNumericalGrade();

- **Arguments:** This function will have zero arguments.
- **Output:** This function will output **one** line to the terminal.
 - "The weighted average based on the entered values is X"
 - X will be a numerical value between 0 and 100 and formatted to two decimal places
- **Return:** A rating between 1 and 5 representing the difficulty of this function.
- **Disclaimer:** You will scan a total of 8 integers (separated by new lines).
- **Disclaimer pt 2:** You must call the required function `outputProcessGrade` after calculating the weighted average. You will pass the average into this function.
- **Disclaimer pt 3:** You must use a different looping structure than the ones used previously.

outputProcessGrade();

- **Arguments:** This function will have one argument.
 - **calculatedGrade:** a double
- **Output:** This function will output the following line:
 - "Your grade based on X will be Y"
 - X will be a numerical value whereas Y will A, B, C, D, F based on X
- **Return:** An single char representing their total grade.

4.4 Shakespeare Email

Context: You're doing this assignment. You are confused or unsure of what to do. Unfortunately, the deadline is X days away and you're trying to decide whether or not to send that email for help. You decide to send that email that same day **but** you know you cannot send an email 24 hours prior. Here is an example with three days

- 3 days prior at 6:00 hours = sends an email wit 90 hours remaining. Input is "3 6"

- 2 days prior at 13:00 hours = sends an email with 59 hours remaining.
Input is "2 13"
- 1 day prior at 23:00 hours = sends an email with 25 hours remaining.
Input is "1 23"

sendEmails();

- **Arguments:** This function will have **two** arguments.
 - **daysLeft:** an integer
 - **hoursLeft:** an integer
 - These will be scanned in values from the user outside of the function on the same line.
- **Output:** This function will output the following **lines** to the terminal:
 - "Sent an email with X hours remaining."
 - X will be the calculated hours left before the deadline
- **Return:** This function will return **nothing**.
- **Disclaimer:** Make sure to not print anything if within the 24 hour period.
- **Disclaimer pt 2:** You must use a different looping structure than the ones used previously.

4.5 Skeleton Code

This is the **required** order of main calling the functions:

```
int main(void)
{
    determineMaxItems();
    determineNumericalGrade();
    sendEmails();

    return 0;
}
```

5 Special Restrictions

You must adhere to the following special restrictions when writing this assignment.

5.1 No Global Variables

You may not use any global variables in this assignment. (A global variable is a variable that is declared outside of a function – e.g., just below a #define statement – and can therefore be accessed by any and all functions in your source file. Example below:

```
int global_variable = 5;

int main(void)
{
    printf("%d\n", global_variable);

    return 1;
}
```

5.2 No File I/O

Your program cannot read or write to any files.

5.3 IDEs

The key to getting multiple files to compile into a single program in Codespaces, Codeblocks, or any IDE is to create a project. Here are the step-by-step instructions for creating a project in CodeBlocks (if you choose to work locally), which involves importing the file you’ve created (even if it’s just an empty file so far).

1. Start CodeBlocks.
2. Create a New Project (File -> New -> Project).
3. Choose “Empty Project” and click “Go.”
4. In the Project Wizard that opens, click “Next.”
5. Input a title for your project (e.g., “Ohce”).
6. Choose a folder (e.g., Desktop) where CodeBlocks can create a subdirectory for the project.
7. Click “Finish.”

Now you need to import your files. You have two options:

1. Drag your source and header files into CodeBlocks. Then right click the tab for each file and choose “Add file to active project.” – or
2. Go to Project -> Add Files.... Browse to the directory with the source and header files you want to import. Select the files from the list (using CTRL-click to select multiple files). Click “Open.” In the dialog box that pops up, click “OK.”

You should now be good to go. Try to build and run the project (F9)

5.4 Linux/Mac Command Line

To compile your source file (.c file) at the command line:

```
gcc continuity.c
```

By default, this will produce an executable file called `a.out`, which you can run with command line arguments like so:

```
./a.out
```

If you want to name the executable file something else, use

```
gcc continuity.c -o continuity
```

...and then run the program with command line arguments like so:

```
./continuity
```

An even better way is combining both commands (by using the Ampersand symbol):

```
gcc continuity.c -o continuity && ./continuity
```

6 General Tips for Working on Programming Assignments

Here's some general advice that might serve you well in this and future assignments in any course:

1. In general, it is ideal to code in short bursts and review your changes early before waiting too long. Example, you check after the completion of a short function or after a few lines in a complex function. The idea is to make sure you are progressively succeeding rather than progressively failing.
2. You're bound to encounter errors in your code at some point. Use `printf()` statements to verify that your code is producing the results you think it should be producing (rather than making assumptions that certain components are working as intended). You should get in the habit of not trusting your own code and using `printf()` to provide yourself with evidence that your code does what you think it does.
3. You will eventually want to read up on how to set break points and use a debugger. Some helpful Google queries might be: CodeBlocks debugger and gdb debugging tutorial.

7 Deliverables (Submitted via Webcourses, not Github Classroom)

Submit your repo's URL via Webcourses. The repo should contain the minimum files for compilation (i.e., the source file, `continuity.c`) and collaboration log.

The source file should contain all the required functions and any helper functions you deem necessary.

Be sure to include your name and UCFID as a comment at the top of your source file similar to how you did codespaces setup.

7.1 Example Output

The following is an example of what your program's output should look like.

```
I can only afford 3 items .
The weighted average based on the entered values is 85.00
Your grade based on 85.00 will be B
Sent an email with 90 hours remaining .
```

8 Grading

The *tentative* scoring breakdown for this programming assignment is:

- 15% determineNumericalGrade() is implemented correctly
- 25% outputProcessGrade is implemented correctly
- 20% determineMaxItems() is implemented correctly
- 20% sendEmails() is implemented correctly
- 10% implementation details (manual inspection of your code)
- 10% adequate comments and whitespace; source includes student name and UCFID

Your grade will be based largely on your program's ability to compile and produce the exact output expected. Even minor changes (such as capitalization or punctuation errors) in your output will cause your program's output to be marked as incorrect, resulting in point deductions. The same is true of how you name your functions and their parameters. Please be sure to follow all requirements carefully and test your program thoroughly.

As a general reminder, please do not wait till the last second to read this or work on the code. It's easy for life to get in the way and especially important to get ahead when given the resources. You got this, good luck!